ACADEMIC PROJECT REPORT

DocCare

Doctor Appointment & Clinical Practice Management System

A Web-Based Healthcare Scheduling Solution with Real-Time Slot Engines & Atomic Double-Booking Prevention

Degree / Program:	**Master of Computer Applications (MCA) / BCA / B.Tech**
Domain:	**Full-Stack Web Development & Healthcare Systems**
Core Technologies:	**PHP 8.x, MySQL, Bootstrap 5, Vanilla JS, AJAX**
Database Name:	**doctor_appointment_system**
Architecture:	**Three-Tier Architecture (MVC Hybrid) with PDO**
Security:	**Bcrypt, CSRF Tokens, Prepared Statements, XSS Escaping**
Local Environment:	**XAMPP (Apache HTTP Server & MariaDB)**

Designed & Developed for MCA Academic Examination • Year 2026

1. Project Abstract & Introduction

1.1 Abstract

In conventional outpatient medical centers, scheduling appointments is frequently conducted via manual paper logs or basic telephone requests. These legacy methods suffer from long patient waiting queues, human booking errors, lack of visibility into doctor shift availability, and inadvertent double-booking of time slots.

DocCare is an enterprise-grade clinical management web application engineered using **Core PHP**, **MySQL**, **Bootstrap 5**, and **JavaScript**. The application features comprehensive **Role-Based Access Control (RBAC)** separating three operational roles: *Patients*, *Doctors*, and *Admin/Receptionists*. A real-time asynchronous JavaScript (AJAX) slot calculation engine queries active bookings and suppresses already occupied or elapsed times. Concurrency-safe atomic database validation eliminates double-booking risks.

1.2 Problem Statement

- **Physical Queue Congestion:** Patients are compelled to visit clinics in person without advance notice of real-time slot occupancy.
- **Double-Booking Vulnerability:** Multiple receptionists or parallel patient submissions often assign the same doctor to two different patients at identical times.
- **Lack of Real-Time Lifecycle Tracking:** Patients remain unaware whether their booking is Pending, Confirmed, Completed, or Cancelled.
- **Disorganized Practice Administration:** Doctors lack centralized oversight over their patient history, daily schedules, and consultation fees.

1.3 Objectives of Proposed System

- To deliver a zero-dependency, professional web portal operating seamlessly on standard XAMPP stacks.
- To automate consultation slot generation according to individual doctor shift timings (start/end times, working days, and slot duration intervals).
- To enforce strict data integrity through PDO prepared statements, input sanitization, and Bcrypt password hashing.
- To provide an intuitive, responsive medical blue-and-white theme accessible across mobile, tablet, and desktop viewports.

2. System Requirements Specification (SRS)

2.1 Software Specifications

COMPONENT	SPECIFICATION DETAILS
Operating System	Microsoft Windows 10/11, Ubuntu Linux, or macOS
Web Server	Apache HTTP Server 2.4.x (via XAMPP) or PHP Built-in Server
Backend Language	Core PHP (PHP 7.4 to PHP 8.5+ compatible)
Database Management	MySQL 5.7+ / MariaDB 10.4+ with InnoDB Storage Engine
Database Admin Tool	phpMyAdmin (Included with XAMPP)
Frontend Framework	Bootstrap 5.3.3 (CSS Grid, Flexbox, Modals, Responsive Navbar)
Client Scripting	Vanilla JavaScript (Fetch API, DOM Event Handlers, Input Filters)
Iconography	Bootstrap Icons v1.11.3 (SVG/WebFont)
Typography	Google Fonts (Outfit for headings, Inter for body copy)

2.2 Hardware Specifications

RESOURCE	MINIMUM REQUIREMENT	RECOMMENDED SPECIFICATION
Processor	Intel Core i3 / AMD Ryzen 3 (1.8 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
RAM Memory	4 GB DDR3/DDR4	8 GB DDR4 or higher
Storage	500 MB Free Disk Space	1 GB SSD Space
Display Resolution	1024 × 768 pixels	1920 × 1080 Full HD (Responsive)

3. System Architecture & Design

The system follows a classic **Three-Tier Layered Architecture** ensuring loose coupling between the presentation layer, application business logic, and persistent database storage:

```
+-------------------------------------------------------------------------------+
|                          1. PRESENTATION LAYER (CLIENT)                       |
```

```
|   HTML5 Semantic Layouts | Custom Medical CSS3 | Bootstrap 5.3.3 | JavaScript     |
|   - Dynamic AJAX Slot Selector Grid & Form Validation Handlers                    |
+-----------------------------------------+-----------------------------------------+
                                          | JSON / HTTP Request
                                          v
+-------------------------------------------------------------------------------+
|                        2. BUSINESS LOGIC LAYER (CORE PHP)                     |
|   - Authentication & RBAC Gatekeeper (`includes/auth.php`)                     |
|   - Shift Generator & Concurrency Engine (`includes/functions.php`)            |
|   - Dynamic API Slot Endpoint (`api/get-slots.php`)                            |
|   - CSRF Validator & XSS Sanitization Filter (`e()`)                           |
+-----------------------------------------+-----------------------------------------+
                                          | PDO Prepared Queries
                                          v
+-------------------------------------------------------------------------------+
|                        3. DATA PERSISTENCE LAYER (MYSQL)                      |
|   Database: `doctor_appointment_system` (InnoDB Engine with Foreign Keys)      |
|   - `users`: Authentication records & role definitions                         |
|   - `doctors`: Clinical schedules, consultation fees, and profile records      |
|   - `appointments`: Booking tickets, timestamps, and lifecycle statuses        |
+-------------------------------------------------------------------------------+
```

4. Database Design & Data Dictionary

The database schema is normalized (3NF) to minimize redundancy while preserving relational integrity via cascading foreign keys and performance-optimized indexes.

4.1 Table: `users`

Stores authentication credentials and contact details for all system actors.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	INT	PRIMARY KEY, AUTO_INC	Unique identifier for the user account
`name`	VARCHAR(100)	NOT NULL	Full legal name of the user
`email`	VARCHAR(100)	NOT NULL, UNIQUE	Unique login email address
`phone`	VARCHAR(20)	NOT NULL	Contact telephone/mobile number
`password`	VARCHAR(255)	NOT NULL	Bcrypt hashed password (60-char hash)
`role`	ENUM('patient','doctor','admin')	NOT NULL, DEFAULT 'patient'	User authorization tier
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp

4.2 Table: `doctors`

Maintains practice parameters, consulting rates, and working schedules.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	INT	PRIMARY KEY, AUTO_INC	Unique doctor profile ID
`user_id`	INT	NOT NULL, UNIQUE, FK → users(id)	Foreign key linking to base user account
`specialization`	VARCHAR(100)	NOT NULL	Clinical discipline (e.g. Cardiologist)

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
qualification	VARCHAR(100)	NOT NULL	Degrees (e.g. MBBS, MD, DM)
consultation_fee	DECIMAL(10,2)	NOT NULL, DEFAULT 500.00	Fee in INR (₹) per patient visit
available_days	VARCHAR(255)	NOT NULL	Comma-separated days (e.g. Mon,Tue,Wed)
start_time	TIME	NOT NULL, DEFAULT '09:00:00'	Daily consultation start time
end_time	TIME	NOT NULL, DEFAULT '17:00:00'	Daily consultation conclusion time
slot_duration	INT	NOT NULL, DEFAULT 30	Duration of each slot in minutes
status	ENUM('active','inactive')	NOT NULL, DEFAULT 'active'	Practice status toggle
bio	TEXT	NULL	Clinical experience summary

4.3 Table: `appointments`

Logs consultation bookings and status history.

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
id	INT	PRIMARY KEY, AUTO_INC	Appointment ticket number
patient_id	INT	NOT NULL, FK → users(id)	Foreign key referencing patient user
doctor_id	INT	NOT NULL, FK → doctors(id)	Foreign key referencing doctor profile
appointment_date	DATE	NOT NULL	Scheduled date of consultation
appointment_time	TIME	NOT NULL	Scheduled start time of slot

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`reason`	TEXT	NOT NULL	Chief complaints / consultation notes
`status`	ENUM('Pending','Confirmed', 'Cancelled','Completed')	NOT NULL, DEFAULT 'Pending'	Lifecycle workflow stage
`booked_by`	ENUM('patient','admin')	NOT NULL, DEFAULT 'patient'	Booking channel originator
`created_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
`updated_at`	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE	Last status change timestamp

5. Core Algorithms & Business Logic

5.1 Real-Time Slot Calculation Logic

When a patient selects a doctor and a consultation date, the frontend triggers an asynchronous `fetch()` request to `api/get-slots.php`. The backend calculates time intervals dynamically:

```
1. Verify doctor status is 'active'.
2. Verify selected_date >= CURRENT_DATE() (Strict past-date rejection).
3. Compute day_of_week = DATE_FORMAT(selected_date, '%W') (e.g., 'Monday').
4. Check if day_of_week exists in doctor.available_days list.
   - If not present: Return message "Doctor is not available on [day]".
5. Query existing active bookings:
   SELECT appointment_time FROM appointments
   WHERE doctor_id = :id AND appointment_date = :date AND status IN ('Pending', 'Confirmed');
6. Iterate time from doctor.start_time to doctor.end_time stepping by (doctor.slot_duration *
60 seconds):
   - IF (date == CURRENT_DATE() AND slot_time <= CURRENT_TIME()):
       mark slot as passed (disabled).
   - ELSE IF (slot_time in booked_slots):
       mark slot as booked (disabled).
   - ELSE:
       mark slot as available (selectable radio button).
7. Return JSON response to client DOM for rendering.
```

5.2 Concurrency & Double-Booking Prevention

Even if two users view the booking page simultaneously, the system prevents race conditions at the database transaction level through the `validate_slot_availability()` function:

```
// Server-Side Pre-Insert Query
$sql = "SELECT id, status FROM appointments
        WHERE doctor_id = ?
          AND appointment_date = ?
          AND appointment_time = ?
          AND status IN ('Pending', 'Confirmed')";
// If row count > 0: Reject transaction with 'Slot Already Booked' error.
```

6. Security Implementation

1. Bcrypt Password Hashing

Passwords are never stored in plaintext. Passwords are encrypted using PHP's native `password_hash($pass, PASSWORD_DEFAULT)`, generating a secure salted Bcrypt hash ($2y$

format). Verification is performed via `password_verify()`.

2. CSRF (Cross-Site Request Forgery) Token Defense

Every form contains an anti-CSRF token generated using cryptographically strong pseudo-random bytes: `$_SESSION['csrf_token'] = bin2hex(random_bytes(32));` Submissions are verified using timing-attack-safe comparison: `hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])`.

3. SQL Injection Neutralization

100% of database interactions are executed using PHP Data Objects (PDO) with prepared statements. Parameter binding isolates SQL instructions from user inputs.

4. XSS (Cross-Site Scripting) Defense

All user-generated values rendered in HTML pages are wrapped in the global escaping function: `function e($str) { return htmlspecialchars((string)$str, ENT_QUOTES, 'UTF-8'); }`.

7. Module Specifications

7.1 Public & Patient Portal

- **Landing Page (`index.php`):** Hero banner, search bar, medical specialties, verified trust counters, and featured doctor cards.
- **Doctor Directory (`doctors.php`):** Search by doctor name or keyword with specialization dropdown filtering.
- **Doctor Profile (`doctor-profile.php`):** Detailed view showing degrees, clinical bio, consultation fee, and direct booking links.
- **Appointment Booking (`patient/book-appointment.php`):** Interactive scheduling form with AJAX-loaded time slot buttons.
- **Appointment Tracking (`patient/my-appointments.php`):** Filterable consultation history with status badges and cancellation actions for pending requests.
- **Profile Settings (`patient/profile.php`):** Update personal contact information and password.

7.2 Doctor Practice Portal

- **Doctor Dashboard (`doctor/index.php`):** Daily schedule, pending requests alert counter, and instant consultation actions.
- **Pending Requests (`doctor/pending-requests.php`):** Review booking details with one-click **Confirm** and **Reject** buttons.
- **Confirmed Schedule (`doctor/appointments.php`):** View upcoming patient contacts and mark appointments as **Completed**.
- **Consultation History (`doctor/history.php`):** Comprehensive consultation logs with date and status filters.
- **Schedule Configuration (`doctor/profile.php`):** Update consulting fees, working days, shifts, and slot lengths.

7.3 Administrator & Receptionist Portal

- **Hospital Control Center (`admin/index.php`):** Live counts of active doctors, patients, today's appointments, and pending requests.
- **Doctor Management (`admin/doctors.php`, `add-doctor.php`, `edit-doctor.php`):** Register new doctors, modify clinical details, and toggle active/inactive status.
- **Patient Directory (`admin/patients.php`):** View patient accounts and booking metrics with quick-booking shortcuts.

- **Master Appointments Log (`admin/appointments.php`):** Filter clinic appointments by doctor, patient, date, and status.
- **Walk-in Booking (`admin/book-appointment.php`):** Receptionist booking for existing patients or on-the-spot registration of walk-ins.

8. Verification & Test Cases Report

An automated test runner (`tests/test_flow.php`) was executed against the database to validate all operational workflows:

ID	TEST SCENARIO	TEST INPUT / CONDITION	EXPECTED RESULT	STATUS
TC-01	Database Connection	PDO connection to MySQL	Active PDO instance established	**PASS**
TC-02	Table Verification	Verify `users`, `doctors`, `appointments`	All 3 tables exist with proper constraints	**PASS**
TC-03	Admin Authentication	`admin@doccare.com` / `admin123`	Password verifies & role is 'admin'	**PASS**
TC-04	Doctor Authentication	`doctor.sharma@doccare.com` / `doctor123`	Password verifies & role is 'doctor'	**PASS**
TC-05	Patient Authentication	`rahul.verma@example.com` / `patient123`	Password verifies & role is 'patient'	**PASS**
TC-06	Doctor Profile Linkage	Check 1-to-1 doctor record for user_id	Specialization, fee & schedule returned	**PASS**
TC-07	Slot Generation	Working day query (e.g. Tuesday)	Time intervals generated (30 min duration)	**PASS**
TC-08	Day-Off Rejection	Non-working day query (e.g. Sunday)	Rejected with "Doctor not available"	**PASS**
TC-09	Past Date Rejection	Date < `CURDATE()`	Rejected with "Cannot book past dates"	**PASS**
TC-10	Double-Booking Check	Second booking on identical slot	Conflict detected; transaction rejected	**PASS**
TC-11	Doctor Confirmation	Status update on Pending appointment	Status changes to 'Confirmed'	**PASS**

ID	TEST SCENARIO	TEST INPUT / CONDITION	EXPECTED RESULT	STATUS
TC-12	Consultation Finish	Status update on Confirmed appointment	Status changes to 'Completed'	PASS
TC-13	Cancellation & Slot Reuse	Appointment marked Cancelled	Slot is freed and open for re-booking	PASS

9. Conclusion & Future Scope

9.1 Conclusion

The **DocCare Doctor Appointment System** successfully demonstrates a complete, reliable, and secure web application for modern medical clinic operations. Built strictly with standard web technologies (Core PHP, MySQL, Bootstrap 5, and JavaScript), it requires no third-party framework overhead, making it lightweight and easy to deploy on local XAMPP environments. The automated slot generation, role-based security, and double-booking prevention resolve core challenges encountered in outpatient healthcare management.

9.2 Future Scope

- **Payment Gateway Integration:** Online payment of consultation fees using UPI, Debit/Credit cards via Razorpay or Stripe.
- **Automated Notifications:** Instant SMS notifications (Twilio) and Email booking confirmations (PHPMailer).
- **Telemedicine Video Consultations:** WebRTC-powered remote video consultations between doctors and patients.
- **E-Prescriptions:** Enabling physicians to generate and attach digital medical prescriptions to completed appointments.